Alternatively, although not necessarily any clearer, the value could be changed using the following method:

```
digitalWrite(13 /*10*/, HIGH);
```

In this example, the value 10 is made into a comment using block comments in the middle of the line, and is, therefore, ignored by the compiler, but leaves it behind just in case you forget the old value. Once the integer value 10 has been hidden from the compiler, the integer 13 will become the new value for this statement. Keep in mind that whenever a section of code is commented out, it will only take effect the next time the sketch is uploaded to the Arduino board.

Line comments work for disabling single lines of code, but it would begin to be quite tedious to place line comments at the beginning of every line in a large block of code that needed to be temporarily ignored. Instead, place block comments around the whole section to be ignored, with `/*` at the beginning and `*/` at the end of the block of code to make the compiler ignore it.

■ **Note** This won't work if somewhere in your block of code you have another set of block comments, as it is not possible to nest block comments inside of one another. In this case, you might have to change the inner block comments to line comments or make some other modifications.

Comments are tremendously useful to the programmer and anyone reading your code, but they can sometimes get in the way, especially if you're trying to write a book. For this reason, after our initial project here, we will no longer comment our code on a line-by-line basis. Yes, that may sound a little like, "do as I say, not as I do," but I hope that it will make the discussion about particular concepts that much clearer without the added verbiage. If necessary to unpack a tricky line, we might add them in there for extra clarity.

Basic Functions

The C programming language is broken down into blocks of code called **functions** that form the main building blocks of any program. In this project we have two functions, `setup()` and `loop()`, each containing statements that do specific actions performed in order from top to bottom. As a requirement of the Arduino library, every sketch must at the very least contain both a `setup()` and `loop()` function in order to operate, even if they are empty. The following is technically a complete and working Arduino sketch, although it doesn't do much:

```
void setup() {
}

void loop() {
}
```

While we can and will create our own functions later in the book to handle certain tasks or consolidate blocks of code, if we intend to use one of these two functions we would need to use both of them. Since empty functions are rather boring, let's look at the following `setup()` function from our first project example: